

Architecture Document — Codity v2

High-Level Architecture

```
flowchart TB
    subgraph Client
        FE[Next.js Dashboard]
        Socket[Socket.IO Client]
    end

    subgraph API_Layer [API Layer]
        Express[Express API Server]
        WS[Socket.IO Server]
        Auth[JWT + RBAC Middleware]
    end

    subgraph Worker_Layer [Worker Layer]
        WM[Worker Manager]
        JC[Job Claimer - SKIP LOCKED]
        JE[Job Executor]
        HB[Heartbeat Service]
        SD[Stale Detector]
    end

    subgraph Data
        PG[(PostgreSQL / Neon)]
    end

    FE -->|REST| Express
    Socket -->|WebSocket| WS
    Express --> Auth
    Express --> PG
    WS --> PG
    WM --> JC
    JC --> PG
    JE --> PG
    HB --> PG
    SD --> PG
```

Job Claiming Sequence

```
sequenceDiagram
    participant W as Worker
    participant DB as PostgreSQL
    participant J as Job Executor

    loop Every 5s
        W->>DB: SELECT ... FOR UPDATE SKIP LOCKED
```

```
DB-->>W: Claimed job IDs
W->>DB: UPDATE status=CLAIMED, locked_until
W->>J: Execute concurrently
J->>DB: status=RUNNING
alt Success
  J->>DB: status=COMPLETED
else Failure + retries left
  J->>DB: status=RETRYING, scheduled_for
else Max retries
  J->>DB: status=DEAD + DLQ entry
end
W->>DB: INSERT heartbeat
end
```

Failure Scenarios

Scenario	Detection	Recovery
Worker crash mid-job	Heartbeat stops	Stale detector marks OFFLINE, releases jobs
Lock TTL expires	locked_until < now()	Job re-claimable by another worker
DB unavailable	Health check fails	API returns degraded; workers retry poll
Duplicate claim	SKIP LOCKED	Prevented at DB level
Idempotent create	idempotency_key unique	Returns existing job

Scaling Strategy

- **API:** Stateless horizontal scaling behind load balancer
- **Workers:** Add instances; each claims independently via SKIP LOCKED
- **Database:** Connection pooling (PgBouncer/Neon), read replicas for analytics
- **Realtime:** Socket.IO with Redis adapter (future) for multi-instance fan-out

Design Decisions

Shared Database vs Message Broker

We use PostgreSQL as the queue store for ACID guarantees, query flexibility, and operational simplicity. Trade-off: ~5-50ms claim latency vs ~1ms for Redis.

Repository Pattern

Prisma access is abstracted into repositories (**JobRepository**, **AuditRepository**) to decouple services from ORM details and enable testing.

Organization Hierarchy

Organization → **Project** → **Queue** → **Job** enables RBAC at org level with **OrganizationMember** roles (OWNER, ADMIN, MEMBER, VIEWER).

Separate Heartbeat Table

High-frequency writes isolated from worker read-heavy dashboard queries.

Audit Logs

Immutable append-only log for compliance and debugging, separate from job execution logs.

Concurrency Strategy

1. **Claiming:** PostgreSQL row-level locks with SKIP LOCKED
2. **Execution:** In-process concurrency limit per worker (semaphore)
3. **Lock TTL:** 5-minute expiry prevents permanent job starvation
4. **Jitter:** Exponential backoff includes random variance to prevent thundering herd

ER Diagram

```
erDiagram
    User ||--o{ OrganizationMember : has
    Organization ||--o{ OrganizationMember : has
    Organization ||--o{ Project : contains
    Project ||--o{ Queue : has
    Project ||--o{ Worker : has
    Project ||--o{ ScheduledJob : has
    Queue ||--o{ Job : contains
    Queue ||--o{ ScheduledJob : targets
    Job ||--o{ JobExecution : has
    Job ||--o{ JobLog : has
    Job ||--o{ DeadLetterQueue : may_have
    Worker ||--o{ WorkerHeartbeat : sends
    User ||--o{ AuditLog : performs
    User ||--o{ Notification : receives
    RetryPolicy ||--o{ Queue : configures
```